

Python Practice

How to use this tab

The exam shows code as **pseudocode**, not Python. Lessons 1–6 use Python because building each structure yourself is the fastest way to understand it; Lesson 7 drills pseudocode tracing directly, the exam's hardest skill.

Priority	Lessons	Why
Do first	7 (tracing), 1 (Big-O, recursion), 2 (lists, stacks, queues), 5 (search and sort)	Cover most of the blueprint
Next	3 (hash tables), 4 (trees and heaps)	Core structures
Optional	6 (graph algorithms)	Mostly beyond this exam

1. Use Python 3.10+ and the starter files (`c949-practice.zip`), or one file per lesson.
2. Type the lesson code, run it, and confirm the printed output matches the comment.
3. Do the exercises before opening Solutions. Each has an `assert` so you know when you're right.
4. For tracing exercises, write a variable table on paper first. Never trace in your head.

Lesson 1: Big-O by experiment

Big-O predicts how runtime grows when n grows. When n doubles, $O(n)$ code takes about twice as long, $O(n^2)$ code takes about four times as long, and $O(\log n)$ code needs only one extra step. Run this code and watch it happen.

```
import time

def time_it(func, n):
    data = list(range(n))
    start = time.perf_counter()
    func(data)
    return time.perf_counter() - start

def linear(data):
    # O(n): one pass
    total = 0
```

```

    for x in data:
        total += x
    return total

def quadratic(data):          # O(n^2): loop inside a loop
    count = 0
    for a in data:
        for b in data:
            count += 1
    return count

def halving(data):           # O(log n): problem halves each step
    n, steps = len(data), 0
    while n > 1:
        n //= 2
        steps += 1
    return steps

for n in (500, 1000, 2000):
    print(n, f'linear={time_it(linear, n):.5f}s', f'quadratic=
{time_it(quadratic, n):.5f}s')
print(halving(list(range(1_000_000))))    # 19

```

In the output, each doubling of n roughly quadruples the quadratic time. A million items need only 19 halvings.

Recursion is a function calling itself. It needs a base case (stop) and a recursive case (smaller problem). Each call takes a stack frame, so deep recursion costs $O(\text{depth})$ memory.

Exercises

1.1 Classify (paper first). Give the Big-O of each snippet.

```

# a
for i in range(n):
    for j in range(10):
        print(i, j)

# b
i = 1
while i < n:
    i *= 2

# c
for i in range(n):
    for j in range(i):
        print(i, j)

# d
for x in list_a:
    for y in list_b:
        print(x, y)

```

1.2 Write `binary_search_max_comparisons(n)` : the most comparisons binary search can need on n items (halve until nothing is left).

```

assert binary_search_max_comparisons(1024) == 11
assert binary_search_max_comparisons(1) == 1
assert binary_search_max_comparisons(1_000_000) == 20

```

1.3 Write the naive recursive `fib(n)` and count how many calls `fib(20)` makes. Then write `fib_memo(n)` using a dict cache so it runs in $O(n)$.

```

assert fib_memo(20) == 6765
assert fib_memo(90) == 2880067194370816120

```

1.4 Write a recursive `sum_list(items)` with no loops. Name its base case.

```

assert sum_list([4, 5, 6]) == 15
assert sum_list([]) == 0

```

Lesson 2: Linked lists, stacks, and queues

A linked list is a chain of nodes connected by `next` pointers. Inserting or removing is just relinking, so it is $O(1)$ once you are at the spot, but finding the spot is $O(n)$. Keeping a `tail` pointer makes append $O(1)$.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def append(self, data):                # 0(1) thanks to tail
        node = Node(data)
        if self.head is None:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node

    def prepend(self, data):              # 0(1)
        node = Node(data)
        node.next = self.head
        self.head = node
        if self.tail is None:
            self.tail = node

    def search(self, key):                 # 0(n)
        cur = self.head
        while cur:
            if cur.data == key:
                return cur
            cur = cur.next
        return None

    def remove(self, key):                 # 0(n): find the predecessor
        prev, cur = None, self.head
        while cur and cur.data != key:
            prev, cur = cur, cur.next
```

```

        if cur is None:
            return False
        if prev is None:
            self.head = cur.next
        else:
            prev.next = cur.next
        if cur is self.tail:
            self.tail = prev
        return True

    def __iter__(self):
        cur = self.head
        while cur:
            yield cur.data
            cur = cur.next

lst = LinkedList()
for x in (2, 3, 4):
    lst.append(x)
lst.prepend(1)
lst.remove(3)
print(list(lst))          # [1, 2, 4]

```

A stack only uses one end (the top), so it can be built as a linked list where push and pop both work at the head. A queue adds at the tail and removes from the head. Every operation is $O(1)$.

```

class Stack:                                # LIFO
    def __init__(self):
        self.top = None
        self.size = 0
    def push(self, data):
        node = Node(data)
        node.next = self.top
        self.top = node
        self.size += 1
    def pop(self):
        if self.top is None:
            raise IndexError('pop from empty stack')
        node = self.top
        self.top = node.next
        self.size -= 1
        return node.data

```

```

def peek(self):
    return None if self.top is None else self.top.data
def is_empty(self):
    return self.top is None

class Queue:                                # FIFO
    def __init__(self):
        self.list = LinkedList()
    def enqueue(self, data):
        self.list.append(data)
    def dequeue(self):
        node = self.list.head
        if node is None:
            raise IndexError('dequeue from empty queue')
        self.list.head = node.next
        if self.list.head is None:
            self.list.tail = None
        return node.data
    def is_empty(self):
        return self.list.head is None

s = Stack()
for x in 'abc':
    s.push(x)
print(s.pop(), s.pop(), s.peek())    # c b a
q = Queue()
for x in 'abc':
    q.enqueue(x)
print(q.dequeue(), q.dequeue())      # a b

```

In real Python code, use `list` with `append / pop` as a stack and `collections.deque` as a queue.

Exercises

2.1 Trace (paper first). What do `s.peek()` and `q.dequeue()` return?

```
s = Stack(); q = Queue()
for x in (1, 2, 3, 4):
    s.push(x); q.enqueue(x)
s.pop(); q.dequeue()
s.push(5); q.enqueue(5)
print(s.peek(), q.dequeue())
```

2.2 Write `reverse(lst)` that reverses a `LinkedList` in place in $O(n)$ time and $O(1)$ extra space. Fix `head` and `tail`.

```
r = LinkedList()
for x in (1, 2, 3, 4):
    r.append(x)
reverse(r)
assert list(r) == [4, 3, 2, 1]
assert r.tail.data == 1
```

2.3 Write `is_balanced(text)` using your `Stack`: true when every `( [ {` has a matching closer in the right order.

```
assert is_balanced('{[()()]}')
assert not is_balanced('([)]')
assert not is_balanced('((')
assert is_balanced('')
```

2.4 Write `eval_postfix(expr)` for space-separated postfix expressions using a stack. Pop the right operand first.

```
assert eval_postfix('3 4 + 2 *') == 14
assert eval_postfix('5 1 2 + 4 * + 3 -') == 14
```

2.5 Build `QueueFromStacks` using two Python lists as stacks. Why is `dequeue` $O(1)$ amortized?

```
qq = QueueFromStacks()
for x in (1, 2, 3):
    qq.enqueue(x)
assert qq.dequeue() == 1
qq.enqueue(4)
assert [qq.dequeue(), qq.dequeue(), qq.dequeue()] == [2, 3, 4]
```

Lesson 3: Hash tables

A hash table turns a key into a bucket index, so a lookup jumps straight to the right bucket instead of searching. Collisions are handled one of two ways: chaining (each bucket is a list) or open addressing (probe for another empty slot). Build both.

```
class ChainingHashTable:
    def __init__(self, size=11):
        self.buckets = [[] for _ in range(size)]
        self.count = 0

    def _index(self, key):
        return hash(key) % len(self.buckets)

    def put(self, key, value):
        # O(1) average
        bucket = self.buckets[self._index(key)]
        for pair in bucket:
            if pair[0] == key:
                pair[1] = value
                # update existing key
                return
        bucket.append([key, value])
        self.count += 1
        if self.count / len(self.buckets) > 0.75:
            self._resize()

    def get(self, key):
        # O(1) average
        for k, v in self.buckets[self._index(key)]:
            if k == key:
                return v
        raise KeyError(key)

    def remove(self, key):
        bucket = self.buckets[self._index(key)]
        for i, (k, _) in enumerate(bucket):
            if k == key:
                bucket.pop(i)
                self.count -= 1
                return
        raise KeyError(key)

    def _resize(self):
        # O(n), but rare
        old = [pair for bucket in self.buckets for pair in bucket]
        self.buckets = [[] for _ in range(len(self.buckets) * 2 + 1)]
```

```

        self.count = 0
        for k, v in old:
            self.put(k, v)

t = ChainingHashTable()
for word in ('apple', 'pear', 'plum'):
    t.put(word, len(word))
t.put('pear', 99)
print(t.get('pear'), t.count)    # 99 3

```

Open addressing stores keys directly in the array. Removal leaves an "empty-after-removal" marker so a later search keeps probing past it; only a never-used slot ends a search.

```

EMPTY_SINCE_START = None
EMPTY_AFTER_REMOVAL = object()

class LinearProbingTable:
    def __init__(self, size=11):
        self.table = [EMPTY_SINCE_START] * size

    def insert(self, key):
        n = len(self.table)
        start = key % n
        for i in range(n):
            j = (start + i) % n
            if self.table[j] is EMPTY_SINCE_START or self.table[j] is
EMPTY_AFTER_REMOVAL:
                self.table[j] = key
                return j
        raise OverflowError('table full')

    def search(self, key):
        n = len(self.table)
        start = key % n
        for i in range(n):
            j = (start + i) % n
            if self.table[j] is EMPTY_SINCE_START:
                return None          # key was never placed past here
            if self.table[j] == key:
                return j
        return None

```

```

def remove(self, key):
    j = self.search(key)
    if j is not None:
        self.table[j] = EMPTY_AFTER_REMOVAL

p = LinearProbingTable()
print([p.insert(k) for k in (20, 31, 42)]) # [9, 10, 0]
p.remove(31)
print(p.search(42)) # 0: probe passes the removed slot

```

Exercises

3.1 Trace (paper first). A 10-slot table uses `key % 10` with linear probing. Which slot does each of 15, 25, 35, 6, 16 land in? Check with `LinearProbingTable(10)`.

3.2 Write `QuadraticProbingTable` (subclass `LinearProbingTable`) that probes `(key % n + i*i) % n`.

```

qp = QuadraticProbingTable(11)
assert [qp.insert(k) for k in (20, 31, 42)] == [9, 10, 2]

```

3.3 Write `string_hash(s, size)` using the polynomial rule $h = h \times 31 + \text{ord}(\text{ch})$, taken mod `size` each step. Why should 'abc' and 'cba' hash differently?

```

assert string_hash('abc', 101) == (((97 * 31 + 98) * 31 + 99) % 101)
assert string_hash('abc', 101) != string_hash('cba', 101)

```

3.4 Write `first_duplicate(items)` in $O(n)$ using a set.

```

assert first_duplicate([3, 1, 4, 1, 5, 9, 3]) == 1
assert first_duplicate([1, 2, 3]) is None

```

3.5 Write `two_sum(nums, target)` returning the index pair in one pass with a dict. What is the brute-force Big-O you're beating?

```

assert two_sum([2, 7, 11, 15], 9) == (0, 1)
assert two_sum([3, 2, 4], 6) == (1, 2)

```

3.6 Insert 100 keys into `ChainingHashTable(3)` and confirm the load factor stays ≤ 0.75 after resizing.

```
big = ChainingHashTable(3)
for i in range(100):
    big.put(i, i * i)
assert big.get(57) == 3249 and big.count == 100
assert big.count / len(big.buckets) <= 0.75
```

Lesson 4: BSTs, traversals, and heaps

A BST keeps smaller keys left and larger keys right, so each comparison discards a subtree. Cost is $O(h)$, the tree's height: about $\log n$ when balanced, n when keys arrive sorted. Removing a node with two children copies in its in-order successor.

```
class TreeNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, key):
        # O(h)
        if self.root is None:
            self.root = TreeNode(key)
            return
        cur = self.root
        while True:
            if key < cur.key:
                if cur.left is None:
                    cur.left = TreeNode(key); return
                cur = cur.left
            else:
                if cur.right is None:
                    cur.right = TreeNode(key); return
                cur = cur.right

    def search(self, key):
        # O(h)
        cur = self.root
        while cur and cur.key != key:
            cur = cur.left if key < cur.key else cur.right
```

```

        return cur

def remove(self, key):
    self.root = self._remove(self.root, key)

def _remove(self, node, key):
    if node is None:
        return None
    if key < node.key:
        node.left = self._remove(node.left, key)
    elif key > node.key:
        node.right = self._remove(node.right, key)
    else:
        if node.left is None:
            return node.right          # 0 or 1 child: splice up
        if node.right is None:
            return node.left
        succ = node.right              # 2 children: in-order
    successor
        while succ.left:
            succ = succ.left
        node.key = succ.key
        node.right = self._remove(node.right, succ.key)
    return node

def inorder(n):    return inorder(n.left) + [n.key] + inorder(n.right) if n
else []
def preorder(n):   return [n.key] + preorder(n.left) + preorder(n.right) if n
else []
def postorder(n):  return postorder(n.left) + postorder(n.right) + [n.key] if
n else []

def height(n):
    if n is None:
        return -1                    # empty tree; a leaf has
height 0
        return 1 + max(height(n.left), height(n.right))

t = BST()
for k in (50, 30, 70, 20, 40, 60, 80):
    t.insert(k)
print(inorder(t.root))    # [20, 30, 40, 50, 60, 70, 80]
print(preorder(t.root))   # [50, 30, 20, 40, 70, 60, 80]

```

```

print(postorder(t.root)) # [20, 40, 30, 60, 80, 70, 50]
print(height(t.root))   # 2
t.remove(50)
print(t.root.key)       # 60: the successor replaced the root

```

A heap is a complete binary tree stored in a plain list. In a max-heap every parent is $\geq$ its children, so the max sits at index 0. Insert appends and swaps up; remove moves the last item to the root and swaps down.

```

class MaxHeap:
    def __init__(self):
        self.a = []

    def insert(self, x):
        # O(log n)
        self.a.append(x)
        i = len(self.a) - 1
        while i > 0:
            parent = (i - 1) // 2
            if self.a[i] <= self.a[parent]:
                break
            self.a[i], self.a[parent] = self.a[parent], self.a[i]
            i = parent

    def remove_max(self):
        # O(log n)
        top = self.a[0]
        last = self.a.pop()
        if self.a:
            self.a[0] = last
            self._down(0)
        return top

    def _down(self, i):
        n = len(self.a)
        while True:
            l, r, big = 2 * i + 1, 2 * i + 2, i
            if l < n and self.a[l] > self.a[big]: big = l
            if r < n and self.a[r] > self.a[big]: big = r
            if big == i:
                return
            self.a[i], self.a[big] = self.a[big], self.a[i]
            i = big

h = MaxHeap()

```

```
for x in (5, 3, 8, 1, 9, 2):
    h.insert(x)
print(h.a)                # [9, 8, 5, 1, 3, 2]
print(h.remove_max(), h.a) # 9 [8, 3, 5, 1, 2]
```

Exercises

4.1 Trace (paper first). Insert 40, 20, 60, 10, 30, 50, 70, 25 into an empty BST. Draw it, then give the preorder list and the height.

4.2 Write `level_order(root)` using `collections.deque` as a queue.

```
assert level_order(tt.root) == [40, 20, 60, 10, 30, 50, 70, 25] # tt = the
tree from 4.1
```

4.3 Write `is_bst(node)`. Checking each node only against its direct children is the classic wrong answer; pass down a (low, high) range instead.

```
assert is_bst(tt.root)
bad = TreeNode(10); bad.left = TreeNode(5); bad.left.right = TreeNode(12)
assert not is_bst(bad)
```

4.4 Write `kth_smallest(root, k)`. Which traversal gives keys in order?

```
assert kth_smallest(tt.root, 3) == 25
```

4.5 Insert 1 through 7 in order into a new BST. What is its height, and what does that do to search cost?

4.6 Trace (paper first). Insert 4, 10, 3, 5, 1 into an empty `MaxHeap`. Write the array after each insert, then confirm five `remove_max()` calls return descending order.

4.7 Write `k_largest(nums, k)` with `heapq` (a min-heap) holding at most k items. Why is this $O(n \log k)$?

```
assert k_largest([3, 1, 9, 7, 5, 8], 3) == [9, 8, 7]
```

Lesson 5: Searching and sorting

The exam may show you an array partway through sorting and ask which algorithm produced it. Writing each sort yourself is the best way to learn what each one looks like mid-sort. Binary search needs sorted input and halves the range every step.

```

def binary_search(a, target):                                #  $O(\log n)$ , a must be sorted
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == target:
            return mid
        if a[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

def bubble_sort(a):                                         #  $O(n^2)$ , stable
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:                                     # best case  $O(n)$ 
            break

def selection_sort(a):                                      #  $O(n^2)$  always
    for i in range(len(a) - 1):
        m = i
        for j in range(i + 1, len(a)):
            if a[j] < a[m]:
                m = j
        a[i], a[m] = a[m], a[i]

def insertion_sort(a):                                      #  $O(n^2)$ ,  $O(n)$  if nearly
sorted                                                       sorted
    for i in range(1, len(a)):
        key, j = a[i], i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key

def merge_sort(a):                                          #  $O(n \log n)$ ,  $O(n)$  extra
    if len(a) <= 1:
        return a

```

```

mid = len(a) // 2
left, right = merge_sort(a[:mid]), merge_sort(a[mid:])
out, i, j = [], 0, 0
while i < len(left) and j < len(right):
    if left[i] <= right[j]:          # <= keeps it stable
        out.append(left[i]); i += 1
    else:
        out.append(right[j]); j += 1
return out + left[i:] + right[j:]

def quicksort(a, lo=0, hi=None):    # avg  $O(n \log n)$ , worst
 $O(n^2)$ 
    if hi is None:
        hi = len(a) - 1
    if lo >= hi:
        return
    pivot = a[hi]                  # Lomuto partition, last item
    as pivot
    i = lo
    for j in range(lo, hi):
        if a[j] < pivot:
            a[i], a[j] = a[j], a[i]
            i += 1
    a[i], a[hi] = a[hi], a[i]
    quicksort(a, lo, i - 1)
    quicksort(a, i + 1, hi)

data = [29, 10, 14, 37, 13]
for sort in (bubble_sort, selection_sort, insertion_sort, quicksort):
    d = data[:]
    sort(d)
    print(sort.__name__, d)        # all [10, 13, 14, 29, 37]
print('merge_sort', merge_sort(data))
print(binary_search([10, 13, 14, 29, 37], 29)) # 3

```

Fingerprints after each pass: selection sort has a growing prefix of the *smallest* items in final position; insertion sort has a sorted prefix that may still change; bubble sort has a growing suffix of the *largest* items.

Exercises

5.1 Trace (paper first). Write the array after each outer pass of selection sort, then of insertion sort, on `[5, 2, 9, 1, 6]`. Add a print inside the outer loop to check.

5.2 Trace (paper first). Binary search for 23 in `[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]`. Which values does `a[mid]` take?

5.3 Write `binary_search_rec(a, target, lo=0, hi=None)` recursively. What is its base case?

```
assert binary_search_rec([1, 3, 5, 7, 9, 11], 7) == 3
assert binary_search_rec([1, 3, 5, 7, 9, 11], 4) == -1
```

5.4 Modify merge sort into `count_inversions(a)` returning `(sorted_list, count)`, where an inversion is a pair $i < j$ with $a[i] > a[j]$. Do it in $O(n \log n)$.

```
assert count_inversions([2, 4, 1, 3, 5])[1] == 3
assert count_inversions([5, 4, 3, 2, 1])[1] == 10
```

5.5 Write `radix_sort(a)` for non-negative ints: bucket by ones digit, then tens, and so on.

```
assert radix_sort([170, 45, 75, 90, 802, 24, 2, 66]) == [2, 24, 45, 66, 75, 90, 170, 802]
```

5.6 Stability: write `merge_sort_key(a, key)` and show records with equal keys keep their order.

```
pairs = [('b', 1), ('a', 2), ('c', 1), ('d', 2)]
assert merge_sort_key(pairs, key=lambda p: p[1]) == [('b', 1), ('c', 1), ('a', 2), ('d', 2)]
```

5.7 Time `quicksort` on 2,000 random floats, then on the same list already sorted (raise the limit with `sys.setrecursionlimit(10000)`). Explain the gap.

Lesson 6: Graphs (optional, mostly beyond this exam)

A Python dict of lists is an adjacency list. BFS and DFS differ only in their container: a queue explores level by level (so BFS finds fewest-edge paths), a stack or recursion dives deep first. Dijkstra is BFS with a priority queue ordered by distance.

```

from collections import deque
import heapq

graph = {                                # adjacency list, undirected
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E'],
}

def bfs(g, start):                        # O(V + E)
    visited, order = {start}, []
    q = deque([start])
    while q:
        v = q.popleft()
        order.append(v)
        for w in g[v]:
            if w not in visited:
                visited.add(w)
                q.append(w)
    return order

def dfs(g, start, visited=None):          # recursion = an implicit stack
    if visited is None:
        visited = []
    visited.append(start)
    for w in g[start]:
        if w not in visited:
            dfs(g, w, visited)
    return visited

print(bfs(graph, 'A'))    # ['A', 'B', 'C', 'D', 'E', 'F']
print(dfs(graph, 'A'))    # ['A', 'B', 'D', 'E', 'F', 'C']

weighted = {
    'A': {'B': 4, 'C': 1},
    'B': {'D': 1},
    'C': {'B': 2, 'D': 5},
    'D': {},
}

```

```
def dijkstra(g, start):          # O((V + E) log V), non-negative weights only
    dist = {v: float('inf') for v in g}
    dist[start] = 0
    pq = [(0, start)]
    while pq:
        d, v = heapq.heappop(pq)
        if d > dist[v]:
            continue             # stale entry, already improved
        for w, weight in g[v].items():
            nd = d + weight
            if nd < dist[w]:
                dist[w] = nd
                heapq.heappush(pq, (nd, w))
    return dist

print(dijkstra(weighted, 'A'))  # {'A': 0, 'B': 3, 'C': 1, 'D': 4}
```

Note $A \rightarrow C \rightarrow B$ ($1 + 2 = 3$) beats the direct $A \rightarrow B$ edge (4). That relaxation step is the whole algorithm.

Exercises

6.1 Trace (paper first). Run Dijkstra by hand from S, listing the distance table after each vertex is popped. Then check with code.

```
w2 = {'S': {'A': 7, 'B': 2}, 'A': {'T': 1}, 'B': {'A': 3, 'T': 8}, 'T': {}}
assert dijkstra(w2, 'S') == {'S': 0, 'A': 5, 'B': 2, 'T': 6}
```

6.2 Write `shortest_path(g, start, goal)` that returns the vertex list of a fewest-edges path. Track each vertex's predecessor during BFS.

```
assert shortest_path(graph, 'A', 'F') == ['A', 'C', 'F']
assert shortest_path(graph, 'D', 'C') == ['D', 'B', 'A', 'C']
```

6.3 Write `dfs_iter(g, start)` with an explicit stack that visits in the same order as the recursive version. (Hint: push neighbors in reverse.)

```
assert dfs_iter(graph, 'A') == dfs(graph, 'A')
```

6.4 Write `count_components(g)` for an undirected graph.

```
islands = {1: [2], 2: [1], 3: [], 4: [5], 5: [4]}
assert count_components(islands) == 3
assert count_components(graph) == 1
```

6.5 Write `topo_sort(g)` (Kahn's algorithm: repeatedly remove vertices with in-degree 0). An edge $X \rightarrow Y$ means X must come before Y . Raise on a cycle.

```
tasks = {'shop': ['cook'], 'cook': ['eat'], 'set_table': ['eat'], 'eat': []}
order = topo_sort(tasks)
assert order.index('shop') < order.index('cook') < order.index('eat')
assert order.index('set_table') < order.index('eat')
```

6.6 Write `to_matrix(g)` returning `(sorted_names, matrix)`. Confirm the matrix for an undirected graph is symmetric. How much memory does each representation use for $V = 10,000$ and $E = 20,000$?

Lesson 7: Tracing pseudocode

This lesson is paper-only: no Python to type. For each drill, draw a table with one column per variable and one row per step, exactly like the worked example in the main guide. Then check your answer in Solutions or in `lesson7.py` from the starter files.

Pseudocode conventions used here: `for i from 1 to 10` includes both 1 and 10; `Put x to output` prints x ; lists start at index 0 unless the drill says otherwise.

Tracing checklist: write the starting values $\rightarrow$ check the condition and write ✓ or ✗ $\rightarrow$ do the body $\rightarrow$ write the new values $\rightarrow$ repeat.

Drills

7.1 While loop. What are x and i when the loop ends?

```
x = 0
i = 1
while i < 10
    x = x + i
    i = i * 2
```

7.2 Branching. What is x at the end?

```

x = 28
if x > 10 and x < 20
    x = 20
else if x < 30
    x = 25
else if x < 50
    x = 100
else
    x = 500

```

7.3 For loop with remainder. What is output?

```

for i from 1 to 10
    if i % 3 == 0
        Put i to output

```

7.4 Nested loops. With $n = 4$, what is count? What is the Big-O of this code?

```

count = 0
for i from 0 to n - 1
    for j from 0 to i - 1
        count = count + 1

```

7.5 Recursion. What does Mystery(7) return? Name the base case.

```

Mystery(n)
    if n <= 0
        return 0
    return n + Mystery(n - 2)

```

7.6 List scan. What is m at the end, and what does this algorithm compute?

```

list = [3, 8, 1, 6]
m = list[0]
for each item in list
    if item > m
        m = item

```

7.7 Heap index math, root at index 1. Index 0 is unused.

```

heapList = [0, 50, 30, 40, 10, 20, 35]

```

What values are the children of 30 (index 2)? What is the parent of 35 (index 6)? Is this a valid max-heap?

7.8 Recursion with output on both sides. What does PrintDown(3) output? (Hint: the second output happens on the way back up.)

```
PrintDown(n)
    if n == 0
        return
    Put n to output
    PrintDown(n - 1)
    Put n to output
```

7.9 Stack operations. What do the two pops return, and what does the final Peek show?

```
Push(s, 5)
Push(s, 2)
Pop(s)
Push(s, 7)
Push(s, 1)
Pop(s)
Peek(s)
```

Solutions

Every solution below was run against its exercise's asserts. Try first; compare after.

Lesson 1

1.1 a: $O(n)$, the inner loop is a constant 10. **b:** $O(\log n)$, i doubles. **c:** $O(n^2)$, about $n^2/2$ iterations. **d:** $O(n \cdot m)$, two different inputs.

```
def binary_search_max_comparisons(n):    # 1.2
    steps = 0
    while n > 0:
        steps += 1
        n //= 2
    return steps

def fib(n):                               # 1.3: fib(20) makes 21,891
    calls, 0(2^n)
    if n < 2:
```

```

        return n
    return fib(n - 1) + fib(n - 2)

def fib_memo(n, memo=None):
    # 0(n): each n computed once
    if memo is None:
        memo = {}
    if n < 2:
        return n
    if n not in memo:
        memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

def sum_list(items):
    # 1.4: base case = empty list
    if not items:
        return 0
    return items[0] + sum_list(items[1:])

```

Trap in 1.4: `items[1:]` copies the list, so this is actually $O(n^2)$. Passing an index instead makes it $O(n)$.

Lesson 2

2.1 `s.peak()` is 5 (pop removed 4, then 5 went on top). `q.dequeue()` is 2 (the first dequeue removed 1).

```

def reverse(lst):
    # 2.2
    prev, cur = None, lst.head
    lst.tail = lst.head
    while cur:
        nxt = cur.next
        cur.next = prev
        prev, cur = cur, nxt
    lst.head = prev

def is_balanced(text):
    # 2.3
    pairs = {')': '(', ']': '[', '}': '{'}
    s = Stack()
    for ch in text:
        if ch in '([{':
            s.push(ch)
        elif ch in pairs:
            if s.is_empty() or s.pop() != pairs[ch]:
                return False

```

```

    return s.is_empty()

def eval_postfix(expr):                                # 2.4
    s = Stack()
    for tok in expr.split():
        if tok in '+-*/':
            b = s.pop()                                # right operand comes off first
            a = s.pop()
            if tok == '+': s.push(a + b)
            elif tok == '-': s.push(a - b)
            elif tok == '*': s.push(a * b)
            else: s.push(a / b)
        else:
            s.push(float(tok))
    return s.pop()

class QueueFromStacks:                                # 2.5
    def __init__(self):
        self.inbox, self.outbox = [], []
    def enqueue(self, x):
        self.inbox.append(x)
    def dequeue(self):
        if not self.outbox:
            while self.inbox:
                self.outbox.append(self.inbox.pop())
        return self.outbox.pop()

```

2.5 is $O(1)$ amortized because each item is moved from inbox to outbox at most once over its lifetime.

Lesson 3

3.1 $15 \rightarrow 5$, $25 \rightarrow 6$, $35 \rightarrow 7$, $6 \rightarrow 8$ (slot 6 is taken), $16 \rightarrow 9$. Five keys, one long cluster: that is primary clustering.

```

class QuadraticProbingTable(LinearProbingTable):      # 3.2
    def insert(self, key):
        n = len(self.table)
        for i in range(n):
            j = (key % n + i * i) % n
            if self.table[j] is EMPTY_SINCE_START or self.table[j] is
EMPTY_AFTER_REMOVAL:
                self.table[j] = key

```

```

        return j
        raise OverflowError('no slot found')

def string_hash(s, size):                                # 3.3
    h = 0
    for ch in s:
        h = (h * 31 + ord(ch)) % size
    return h

def first_duplicate(items):                              # 3.4
    seen = set()
    for x in items:
        if x in seen:
            return x
        seen.add(x)
    return None

def two_sum(nums, target):                              # 3.5
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return (seen[target - x], i)
        seen[x] = i
    return None

```

3.3: summing character codes would give every anagram the same hash; multiplying by 31 makes position matter. 3.5 beats the $O(n^2)$ check of every pair. 3.6 needs no new code: the table resizes from 3 to 255 buckets along the way.

Lesson 4

4.1 Root 40; left 20 (children 10 and 30, with 25 as 30's left child); right 60 (children 50 and 70). Preorder: 40, 20, 10, 30, 25, 60, 50, 70. Height 3 (40 → 20 → 30 → 25).

```

from collections import deque

def level_order(root):                                  # 4.2
    out, q = [], deque([root] if root else [])
    while q:
        n = q.popleft()
        out.append(n.key)
        if n.left: q.append(n.left)
        if n.right: q.append(n.right)

```

```

    return out

def is_bst(n, lo=float('-inf'), hi=float('inf')):    # 4.3
    if n is None:
        return True
    if not (lo <= n.key < hi):
        return False
    return is_bst(n.left, lo, n.key) and is_bst(n.right, n.key, hi)

def kth_smallest(root, k):                        # 4.4: inorder = sorted
    return inorder(root)[k - 1]

import heapq
def k_largest(nums, k):                          # 4.7
    heap = []
    for x in nums:
        heapq.heappush(heap, x)
        if len(heap) > k:
            heapq.heappop(heap)                  # drop the smallest of the k+1
    return sorted(heap, reverse=True)

```

4.5 Height 6: every node has only a right child, so the tree is a linked list and search is $O(n)$.

4.6 $[4] \rightarrow [10, 4] \rightarrow [10, 4, 3] \rightarrow [10, 5, 3, 4]$ (5 swapped up past 4) $\rightarrow [10, 5, 3, 4, 1]$. Removals return 10, 5, 4, 3, 1.

4.7 The heap never holds more than k items, so each of the n pushes and pops costs $O(\log k)$.

Lesson 5

5.1 Selection: $[1, 2, 9, 5, 6] \rightarrow [1, 2, 9, 5, 6] \rightarrow [1, 2, 5, 9, 6] \rightarrow [1, 2, 5, 6, 9]$. Insertion: $[2, 5, 9, 1, 6] \rightarrow [2, 5, 9, 1, 6] \rightarrow [1, 2, 5, 9, 6] \rightarrow [1, 2, 5, 6, 9]$.

5.2 16 (index 4), then 56 (index 7), then 23 (index 5): three comparisons.

```

def binary_search_rec(a, target, lo=0, hi=None):    # 5.3: base case lo > hi
    if hi is None:
        hi = len(a) - 1
    if lo > hi:
        return -1
    mid = (lo + hi) // 2
    if a[mid] == target:

```

```

        return mid
    if a[mid] < target:
        return binary_search_rec(a, target, mid + 1, hi)
    return binary_search_rec(a, target, lo, mid - 1)

def count_inversions(a):                                # 5.4
    if len(a) <= 1:
        return a, 0
    mid = len(a) // 2
    left, x = count_inversions(a[:mid])
    right, y = count_inversions(a[mid:])
    out, i, j, inv = [], 0, 0, x + y
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
            inv += len(left) - i                # right[j] jumps every remaining
left item
    return out + left[i:] + right[j:], inv

def radix_sort(a):                                    # 5.5: LSD, O(nk)
    exp = 1
    while a and max(a) // exp > 0:
        buckets = [[] for _ in range(10)]
        for x in a:
            buckets[(x // exp) % 10].append(x)
        a = [x for b in buckets for x in b]
        exp *= 10
    return a

def merge_sort_key(a, key):                            # 5.6
    if len(a) <= 1:
        return a
    m = len(a) // 2
    l, r = merge_sort_key(a[:m], key), merge_sort_key(a[m:], key)
    out, i, j = [], 0, 0
    while i < len(l) and j < len(r):
        if key(l[i]) <= key(r[j]):                # <= takes the left item on ties
            out.append(l[i]); i += 1
        else:
            out.append(r[j]); j += 1
    return out + l[i:] + r[j:]

```

5.7 Roughly 0.002 s random vs 0.18 s sorted. With the last item as pivot, sorted input splits every partition into $n - 1$ and 0, so quicksort hits its $O(n^2)$ worst case and recursion goes n deep. Median-of-three or a random pivot fixes it.

Lesson 6

6.1 Pop S: A = 7, B = 2. Pop B: A = 5 (2 + 3), T = 10. Pop A: T = 6 (5 + 1). Pop T: done.

```
def shortest_path(g, start, goal):          # 6.2
    prev = {start: None}
    q = deque([start])
    while q:
        v = q.popleft()
        if v == goal:
            path = []
            while v is not None:
                path.append(v); v = prev[v]
            return path[::-1]
        for w in g[v]:
            if w not in prev:
                prev[w] = v; q.append(w)
    return None

def dfs_iter(g, start):                    # 6.3
    visited, stack = [], [start]
    while stack:
        v = stack.pop()
        if v in visited:
            continue
        visited.append(v)
        for w in reversed(g[v]):
            if w not in visited:
                stack.append(w)
    return visited

def count_components(g):                  # 6.4
    seen, count = set(), 0
    for v in g:
        if v not in seen:
            count += 1
            seen.update(bfs(g, v))
    return count
```

```

def topo_sort(g):                                     # 6.5: Kahn's algorithm
    indeg = {v: 0 for v in g}
    for v in g:
        for w in g[v]:
            indeg[w] += 1
    q = deque(sorted(v for v in g if indeg[v] == 0))
    order = []
    while q:
        v = q.popleft(); order.append(v)
        for w in g[v]:
            indeg[w] -= 1
            if indeg[w] == 0:
                q.append(w)
    if len(order) != len(g):
        raise ValueError('cycle')
    return order

def to_matrix(g):                                     # 6.6
    names = sorted(g)
    idx = {v: i for i, v in enumerate(names)}
    m = [[0] * len(names) for _ in names]
    for v in g:
        for w in g[v]:
            m[idx[v]][idx[w]] = 1
    return names, m

```

6.6 Matrix: $V^2 = 100,000,000$ cells. List: $V + 2E = 50,000$ entries (each undirected edge is stored twice). Sparse graphs belong in adjacency lists.

Lesson 7

7.1 $x = 15, i = 16$.

i (check)	x after body	i after body
1 < 10 ✓	1	2
2 < 10 ✓	3	4
4 < 10 ✓	7	8
8 < 10 ✓	15	16
16 < 10 ✗	stop	

7.2 $x = 25$. 28 fails the first test (not < 20), passes the second (< 30), and the rest are skipped. Only one branch of an if/else if chain ever runs.

7.3 Output 3, 6, 9. The range includes 10, but $10 \% 3$ is 1.

7.4 $\text{count} = 6$. The inner loop runs 0, 1, 2, then 3 times ($0 + 1 + 2 + 3$). That's about $n^2/2$ steps, so $O(n^2)$.

7.5 16. $\text{Mystery}(7) = 7 + \text{Mystery}(5) = 7 + 5 + \text{Mystery}(3) = 7 + 5 + 3 + \text{Mystery}(1) = 7 + 5 + 3 + 1 + \text{Mystery}(-1) = 16$. The base case is $n \leq 0$, which returns 0.

7.6 $m = 8$. It finds the maximum value in the list. It's $O(n)$, one pass.

7.7 Children of index 2 are indexes 4 and 5 ($2i, 2i + 1$): values 10 and 20. Parent of index 6 is index 3 ($6 // 2$): value 40. Yes, it's a valid max-heap: $50 \geq 30, 40$; $30 \geq 10, 20$; $40 \geq 35$.

7.8 3 2 1 1 2 3. Each call prints on the way down, waits for the smaller call to finish, then prints again on the way back up.

7.9 The pops return 2, then 1. Peek shows 7, and the stack holds 5 (bottom) and 7 (top).